Research Report: Best packaging and installer solutions for Python desktop applications with PyTorch CUDA GPU dependencies targeting Windows macOS and Linux - one-click installer for non-technical users, handling large model weights (5-10GB), auto-updates, no CLI or conda required

Generated: 03/12/2026, 17:33:45
Sources: 163 verified

Executive Summary

The most effective solution for packaging a PySide6 and PyTorch/CUDA desktop application for non-technical users is a hybrid architecture using an **Electron frontend with a Nuitka-compiled Python backend**, packaged by **electron-builder**. This approach provides a modern cross-platform UI, robust auto-updates for the core application, and the necessary customization to handle complex dependencies and large files. For the 5-10GB model weights, they must be managed separately from the main application installer and downloaded on the first launch from within the app.

This strategy directly addresses the core challenges. Nuitka provides significant performance benefits, with cold-start times of 2-5 seconds, compared to 5-10+ seconds for PyInstaller's user-friendly single-file mode [3]. The CUDA runtime must be bundled directly from the official PyTorch wheel, which adds 500-800MB to the package size but is the only reliable method for a one-click install [6]. Auto-updates require a two-part strategy: `electron-updater` handles differential updates for the application binary, while a custom in-app solution using `zsync` is required to manage differential updates for the multi-gigabyte model files, as other methods like `bsdiff` have prohibitive client-side memory requirements [4, 7].

Key Findings

1. Finding 1: The optimal architecture is a hybrid of an Electron frontend and a Nuitka-compiled Python backend, packaged with `electron-builder`. This provides the best balance of modern UI, cross-platform one-click installers, and robust auto-updates via `electron-updater`, which is critical for

Confidence: HIGH
Sources: Multiple sources

2. Finding 2: Large model weights (5-10GB) must be downloaded on the application's first launch, not bundled in the installer. This provides a better user experience with progress reporting, reliable error handling (e.g., pause/resume), and avoids installer timeouts and corruption issues.

- *

Confidence: HIGH
Sources: [2]

3. Finding 3: Auto-updates require a two-part strategy. `electron-updater` can only manage differential updates for the core application package. A separate, custom-built process is required to check for and download updates to the large model files.

- **Confidence:** HIGH
- Sources:*

Confidence: HIGH
Sources: Multiple sources

4. Finding 4: Nuitka significantly outperforms PyInstaller in cold-start time for complex applications. By compiling to native C code, Nuitka achieves startup times of 2-5 seconds, while PyInstaller's single-file mode can take over 10 seconds due to the need to decompress all libraries on every lau

Confidence: HIGH
Sources: [3]

5. Finding 5: Bundling the CUDA runtime is non-negotiable for a seamless user experience. The official PyTorch wheel includes the necessary libraries, and Nuitka will bundle them. This adds 500-800MB to the final package but eliminates the need for users to install a system-wide CUDA toolkit, which

Confidence: HIGH
Sources: [6]

6. Finding 6: For implementing differential updates on the multi-gigabyte model files, `zsync` is the most practical technology. Unlike `bsdiff`, which can require massive amounts of client-side RAM (e.g., 17x the file size), `zsync` has low memory requirements and works over standard HTTP servers.

Confidence: LOW
Sources: Multiple sources

7. Finding 7: A local web server using a framework like FastAPI is the most robust method for communication between the Electron frontend and Python backend. It offers superior performance and stability for long-running, intensive tasks like video processing compared to shell-based methods.

-

Confidence: LOW
Sources: [5]

Detailed Analysis

Packaging & Installer Framework Comparison

For the target audience of non-technical VFX artists, the installation and update experience is paramount. While pure Python solutions like PyInstaller, Nuitka, and cx_Freeze can create executables, they lack a mature, built-in

framework for auto-updates and creating polished, cross-platform installers (e.g., `.dmg` on macOS, NSIS on Windows).

The recommended solution is a hybrid architecture using **Electron** for the frontend and UI, which communicates with a Python backend process [1]. This backend should be compiled into a standalone executable using **Nuitka** for performance. The entire application is then packaged using **electron-builder**.

- **Electron + electron-builder**: Provides a best-in-class user experience. It generates true one-click installers for Windows (NSIS), macOS (`.dmg`), and Linux (`.AppImage` , `.deb`). Crucially, it integrates with `electron-updater` , which provides seamless, silent auto-updates out-of-the-box [1, 4]. The UI, built with web technologies, remains responsive even when the Python backend is under heavy load. The primary tradeoff is a higher baseline memory footprint (100-150MB+) due to the bundled Chromium instance, but this is acceptable given the memory-intensive nature of the core task [3].

- **Nuitka**: Compiles Python to C, resulting in a faster cold-start (2-5 seconds) and potentially faster execution of CPU-bound code. This is a decisive advantage over **PyInstaller** in `--one-file` mode, which can take 5-10+ seconds to start because it must decompress the entire application into a temporary directory at every launch [3]. Nuitka's PyTorch plugin is designed to correctly trace and bundle the complex dependencies of the library, including its CUDA components [6].

- **Briefcase / cx_Freeze**: While viable, these tools do not have the same level of community support and maturity for handling all the specified requirements (especially auto-updates and complex C-extension bundling) as the Electron/Nuitka combination.

Handling Large Model Weights

Bundling 5-10GB of model data into a single installer file is impractical. It leads to a poor user experience with extremely long download times for the initial install, and it complicates the update process. The installer itself is likely to be flagged by antivirus software and is prone to corruption during download [2].

The standard and most robust method is to **download the models on the first launch** [2]:
1. The initial installer is small, containing only the core application logic.
2. On first run, the app checks for the presence of the model files in a user-accessible data directory (`app.getPath('userData')`).
3. If the models are missing, the Electron UI displays a dedicated download manager window, showing progress, speed, and time remaining.
4. The download is handled in the Node.js main process using a library like `axios` to fetch the files from a CDN. This allows for features like pausing and resuming.
5. Once the download is complete and files are verified (e.g., with a checksum), the main application window is launched.

GPU & CUDA Dependency Management

For a one-click installer, the application must be self-contained and cannot rely on the user having a specific version of the CUDA toolkit installed system-wide. The official PyTorch wheels for Windows and Linux come with their own embedded CUDA runtime libraries (e.g., `cudart64_*.dll` , `lib` files) [6].

The packaging process must ensure these files are included:
1. During development, install the CUDA-enabled version of PyTorch (e.g., `pip install torch --index-url https://download.pytorch.org/whl/cu121`).
2. Use Nuitka with its PyTorch plugin (`--enable-plugin=torch`) to compile the Python backend. The plugin is specifically designed to find and bundle all necessary files from the PyTorch wheel, including the CUDA and cuDNN libraries [6].
3. The final Nuitka output (a folder or single executable) will contain these libraries. This compiled backend is then included in the Electron app as an "extra resource" and launched by the main process [5].
4. On macOS, PyTorch uses the Metal Performance Shaders (MPS) backend, not CUDA. The build process must account for this, packaging a backend that uses the MPS-enabled PyTorch wheel for the macOS target.

Auto-Update Strategy

A two-part strategy is required to handle updates for the application and the large models separately.

1. **Application Updates**: `electron-updater` handles this automatically. When `electron-builder` creates a release, it also generates a blockmap file. The client application periodically checks a release server (e.g., GitHub Releases), downloads the blockmap, compares it to the currently installed version, and downloads only the changed blocks. This provides efficient, differential updates for the core application binary [4].

2. **Model Updates**: `electron-updater` cannot manage these external files [4]. A custom solution must be built into the application. To provide differential updates and save users from re-downloading 10GB, `zsync` is the recommended technology. Unlike `bsdiff`, which would require an impractical amount of RAM on the user's machine to patch a 5GB file, `zsync` uses a rolling checksum and HTTP range requests to download only the changed parts of a file. The workflow is:
- Host the model files and a corresponding `.zsync` file (generated server-side) on an HTTP server.
- The application periodically checks a manifest for new model versions.
- If an update is found, the app uses a `zsync` client implementation to update the local model file, minimizing data transfer [7].

Recommendations

1. **Adopt the Hybrid Architecture**: Use Electron for the frontend UI and package the application with `electron-builder`. Compile the core Python/PyTorch logic into a standalone executable using Nuitka to maximize performance and simplify integration.
2. **Separate Application and Model Data**: Create a small initial installer that contains only the application. Implement a mandatory, user-friendly download manager that runs on first launch to download the 5-10GB of model weights to the user's data directory.
3. **Use `electron-updater` for App Updates**: Leverage the mature, built-in auto-update framework provided by `electron-builder` and `electron-updater` for seamless and efficient updates to the core application.
4. **Implement `zsync` for Model Updates**: To provide differential updates for the multi-gigabyte model files, build a custom in-app updater based on `zsync`. This is the only practical approach that respects user bandwidth and avoids extreme client-side memory consumption.
5. **Use a Local Web Server for IPC**: For robust communication between the Electron frontend and the Nuitka-compiled backend, run a lightweight web server (e.g., FastAPI) within the Python process. This is more stable and performant for intensive, long-running tasks than standard I/O piping.

Limitations & Caveats

- **Complexity**: The recommended hybrid architecture is more complex to develop and maintain than a monolithic application built with a single framework. It requires expertise in both the Node.js/Electron ecosystem and the Python data science stack.

- **Custom Update Logic**: `electron-updater`'s inability to handle external assets means you must design, build, and maintain a custom and non-trivial update mechanism for the large model files.

- **Platform Divergence (CUDA vs. MPS)**: The requirement to support both NVIDIA (CUDA) on Windows/Linux and Apple Silicon (MPS) on macOS adds significant complexity to the build and testing pipeline. The application backend will need conditional logic to utilize the correct device, and the build process must package different PyTorch dependencies for each platform.

- **IPC Overhead**: While a local web server is robust, it introduces a small amount of latency and overhead compared to direct in-process calls. The confidence in this specific recommendation is lower, as simpler

methods may suffice depending on the exact nature of the UI-backend interaction.

Follow-up Questions

1. What is the detailed build and CI/CD pipeline for creating three separate artifacts (Windows-CUDA, Linux-CUDA, macOS-MPS) that correctly bundle their respective Nuitka-compiled backends within an Electron app?
2. Are there existing open-source or commercial libraries that simplify the client-side implementation of `zsync` within a Node.js/Electron environment?
3. For the specific video processing workload, what is the real-world performance and memory overhead of a FastAPI-based IPC versus a simpler `python-shell` (stdin/stdout) approach?
4. How can the application gracefully detect and guide users who lack the required hardware (e.g., a compatible NVIDIA GPU on Windows or an M-series Mac)?

Sources

[1] google.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQED-p6wqtijvpepfgdUMzaSWnBXVQCHw4tC6oGl_0DAGf48JMp4SHDuwOGGX9AZy3SKAmB-fhZ5qZYtPz2FpYuyCEHUmlk-vMXiWEEgLovvR6dfQMnwbUwTMOkvuuhEGP1OABeQ1fD3OyxFl0kqx60-kMfE-zcR4yEMav3Ro4rqwg==
[2] microsoft.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFfR8LBGvE1RgpAnqck0h2GDxdkiFNkqSttuPXKyr7A-AfwFFquhghlpQN1DhNGTHruGSokExat8Ut8bqwss8P-Eoy4y2N0lLLhMEUOsjo0stkz-CMq9qtl6344nW2IqfVxMVu15xdsIr8=
[3] apple.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHFdvwEv2JAHxpP5dep2J6bx64r1kIEHAX0ooVVacEEfCbbFga8Ff4ArQ67yKZh_tcFSj4zH_4Y0XvV1pU8G46dGlT
IKUuVFwpw6BCsvnTb-2EqwGzoTg5EdEhLIXhXuyWF_UM6uKpmdTrwSBr6-E_X1QEvGZi_VwlpbGqDNcnQ2ebQAXL0l6ax60viNPy7w8gxI9fQL9iN8MKQvD-e81
[4] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGJ6sBFvLS00N5UDEhyxmK9rOULgaegYpnf14R--Zb41Z201mUbwvOPInqYTc7_yBMJX78gw-VEmTIRSfSY2nFHH7OJduJ_tVUl6o8bsxW8nJLKhqqjIbCGI8AhofCjU61-UtmPe-0=
[5] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHxx3E46eLzqDTA5CcgE_FEoChBMy0Hy6Q98XEYQXFfhhGnk8Pm7iwYS1v4h63AyeKWDWPWexTjDexvvvocEiy
[6] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHyuy8wp2xgGmoqeSkZXDrSJuzzMmqczvSdwDJ_7zscVqNKpCis3IytOxnP-xrhcAwFJMjYrBtWMdOHcBU4_NXNIxAn2Ilru5PcsyLyR3I6JdJfuGIZJX4sDJQ2KdGO2EiI0j31SvI=
[7] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGqMKKlLdGnm_L4rTUEIhvwL7Y4B0bxOBK5Lqk9bffnQBnvFKsb-PIh9Wi_W5VwYtYgpzdzp2pwo-nhaGZIVXInb1P7aXm0TTyoTBSxTG-3VZoEEJil7q4Bcli5PbEUcbxWt2iA1ZM=
[8] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFYQDE_N17ZzIAGIYOGiLTs-9mKqPx_-3_sYd112RVkN9BQzjAqjrvnkrYihEUj5RXQA8UrXT_dLc1PIMmd5XyH8sQ4wS0I8vBPD4cH1sdjND0jn6AOc91CuiQ3LOd5
[9] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGtPIYeNjzygbfG2Fdke37xHT1af6Tp4WTZ8lW5d-B1dPzkGKGnG_7XvWwe7rfCdWv6xgi6YUrB81FsPtjbpwpVcer8lhwZxwk8wPr83CeSFw3_x0-x9k3DNOUwApPxhcR0huFFIA==
[10] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEz6b1i6QTqRBwo7XonQcgP_dhU0aoiRU0y0wkZ9et7KJfNu_eoJ-nn19CP0x4QABpaguvjxIuuNS2DxgsfZphtHU8AeNazBSChqjxT0SrEg_8HDRANEFHm0qIEqYZF5SkEhDrB0BI=
[11] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHQI8eOJttQ5kuAJJUUg5OsjrS4ILNNbLzz8AvCQvc3wLS8GE2jPy75tungXwE-VZLLJjSJQ6S-3vxLRNsJ2Xta8j8uDUghnBykI1KPmF0MoVZjVwNeMooiUYNKaEKbSo5dM77p2ME=
[12] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGe9baWqf_guri4A11dOUAuXaBQoiOxbBdhVfV_zA-_dJAZQqtAuwSU-2QCo4QrD2bIwSja5E2rEqIc833wEz-Q1cMzryw5jk-qvysGN7FhFweMQjqfNDvUAiTNHMC0IY5-WhYWEg==

[13] youtube.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFCSiT2RzGWqvuR8scxg2ewv1YtiSqIh8I6L1sFQDyV7Kwh_MME9w2d-MNSsdi0AYtehCkU31DqORnGyfqKpXW_5KDUO_HqYEiCdT_X1jJo9YIpyWIVpAMLt2YqxpZ2VEIWJbgejKI=
[14] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGeW7EpkW3UGqTU_PwiNdIzOrtGWQIskLMg9eEo-e4TVCnHVndQQCZUCXLVyqZQWdWg5pC705Zs_xqQeqVqxyGz35ofWsdaF3UN0xv-RAeDd1TMVCsGPiRVmKcitq_viIzzY0sxPV3ODXQLLIpyLhuoXfv1Jkz4ow==
[15] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGJQkzVmH6Q9opc1fQ-KOCyNFwxTihrTUJ2X1tEhdYigSWIPEVID2ndL95ZbKa0-vomRu4iCNJYya7sglO1INBquX871FSaAM9cZyNu-fxR0zt-UuGL_550GL1VAYPZQkNdeA_7F7A8ptHFdQNbDLMY6B_qU4NO5QZVI8I=
[16] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFzfVUQQJXDYmUk9t52N175MHxNNCfUIQy5RVIvEoWRG71gMmwiX0AKfTXa2ddn5zr90iguUIZXcVBQkPIUITUi4
ksqwvd1xTaeHE468=
[17] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHFjUCPFaIEaP1MV2V3AgJ3_sWW73KVd5992hFWjejpLtNy9fKtBC7TodcRmdYUcrYpO4P74sWOcY_icd9TIGV9w
[18] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFpsj_goe7fuCn6I8vAaoEeilk7zIN86L3cCAUIr_fufQh3hOaEkjn3oVZ-PS_hd2ChyfewnznUbCOC4f1TOb998Akd78f-E3of5wS92uIcUnEsRzbtbvRJg4nFkkxU9jkU3hEkP3MfP9Np7tVFIQ==
[19] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFhNL-oLrxJtO3ncuFRMgSM8Zv2NUZE94NiBc-qFcQvgIyk0LqnqZo43-9r4C1dHuR_I456uiQzjFXBhwMu7yz5LUOGyDUIsF97TPFp9-o7GFPeTTq_nxewO0W3jjko4hoN0fRzCChqJKkeNDNIdA6BODKUVWjLxuFooN7P
[20] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGFXmepM551Cup4X4G9hsAAROUnNReURDq_sr4_6YuvYI0CZOYdb0137WIJjBVPTRiuz-CLURxGfzftdLdSObjxKeyHgio9Ij8zAg0hgxcetx6Z9Tg2fj9SX_PaWB-VXWxckkL-bGf7-ZEgOnwaL46HtZXWsshonDttM5s=
[21] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGbzPsqBTXNM_dZt-2kMivn8CdZtqkCdIO2fUREYHiTDh4BCKWDkf27q-uqs2gIBRcvEQttITI9TW3qzZdrvDQ9t3z_PwcTj-18cakDsftgm7CVJTyi31PHQSyvilDF8WOHr_CPTdPagYRcHyX8okvHDr8FZ7NeWb6TX9k=
[22] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQGqn9KveXIFv-sAPaGps4qay7jol1pD5LZMDSzyNHNbP7GXRCtSYLBi14Y4gSMiF6s6o4w9SWT4BSFsfDSD3UgXoNh_BwsC5MJ_hfa8XEkn8W0s
UUTFYy_-yiytT7Zitu0wFEwpFIMf6mpMjRS8SI7T7A5Cbbc=
[23] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQFLtsp5Q1QIm4i4nBNrJ7-ZP8SvFH88reg2EyyCFCS4kCnQgildLxzL9etX6YRmiEFVV8TSmyxsJmfBAiHOD5VNQQNaPMN2EIQPsR7UzzHKEApM1JSGIgSFm
r7LLRuaskwwYYm7EN-XGbeXqf
[24] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEyx9wAOa4KQwgVWPNVB12VQ_yXubEtyeqkcA3G_pkysdz2oGCAUtQGZS5dBvbFWIRgCjXci5Z4ObEWkChJ4fx
F9N1O
[25] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQG-XSbjgM3V_14-ugpE8G_u_ZvG5EaVcAIHcOJ-g94PGDS7vB5yY2ReRlpSaU6zvzsMX-GL1pBiHj_AJrmq14iy49dNoW3tpDyAC2XyGUdL9P_MTqvWzRwYW3IvvnVi
[26] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEFq-GupvASIpYgYFJwjsSavR8XAAGX7IdCF8EqiKHkxnb7dK7QvA7sipz245-Df7m_7yUMa_7gMUybBcU014NgzBCTUJ4rt82X1pQNsKBUGs-Xs8SjZqr0UH_h3ZEbvrFTpq89-ZQBXwqXxjq-2LQnLBjrIhvIaUcId_o=
[27] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQEww-V6utM0pqxWtzasYKiHxwS9LVmjrQQRwwjJ47i9QAduFw6omHJntRawuSKyiWoAAMK8RBBcL-0YcXf4UfhGh_Ki74FnRgmPGhLeSnjaqoI04XN9X0eTG1_u0BPnLLM2zHIz_kVQ4iyY8zK706×6CSHoMpfqWEArWZs=
[28] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQHzc-kHu_k-SpkazAXvBzkOduwb6XEPjpPTX0ijJUmn8-jEA66aIDiwAQkDPBqHi_RNDBJ6o29p-R7csjhtPdFrEbU349sH0ULoXW2GZt01G6wNmB4rCWUYd5oaD4-k3HF4GIjgOhk6-VI=
[29] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQF2IW6xkBnKLIDGbm_3S3Hs4QJ2Z2JBqpQSG-bo53VnfsDhxMaZ6RMWUFxJjTiFUqFkZCO3HuNCuIoOp6bWjbpwBpmFk5F_iCyhM2f_5PgCowx1p9LX-Te-RTuH-AWG1vw73nA4bg==
[30] github.com. https://vertexaisearch.cloud.google.com/grounding-api-redirect/AUZIYQECwpKrzxUBTVYOfGTK3DXY33w-

1oY8wcNMTCJjS0poPjq2l_7dVBbtNkrMpFYWVjocDr5MKcki3AD-JYGeE3cEMrljlvLE9H2gv4grjmvUM-0w5xJoor7OmvBs4gQPv8JRvqoSO7I=

Methodology

- **Backend**: VERTEX API
- **Model**: gemini-2.5-pro
- **Research Depth**: 4 (follow-up iterations)
- **Research Breadth**: 4 (parallel queries per iteration)
- **Total Sources Evaluated**: 163
- **Quality Threshold**: 40%+ (verified sources only)
- **Duration**: 732.8s
- **Synthesis Method**: AI-assisted cross-referencing and verification

---

API Costs

Metric	Value
Input Tokens	133,643
Output Tokens	20,435
Total Tokens	**154,078**
Input Cost	$0.1671
Output Cost	$0.2043
Total Cost	**$0.3714**

Report generated by ez-deep-research MCP